Lab3 AttackLab 实验报告

姓名：施展鸿　学号：25300120066　日期：2026-10-28

实验环境

目标二进制：attack/dark-calc（非 PIE，固定地址栈，栈可执行）、attack/dark-calc-nx（非 PIE，NX + ASLR）、attack/dark-log（非 PIE，NX + Stack Protector）。调试工具：gdb 17.1（WSL2 Ubuntu 26.04）；反汇编：objdump；payload 由仓库根目录的 payload.py / payload_dark.py / payload_rop.py / payload_canary.py 生成。

一、任务：观察栈帧 input

用 gdb ./dark-calc 在 input 处下断点（b input），输入一个普通表达式后断点命中于 0x40197f。GDB 输出如下：

```
#0  0x000000000040197f in input ()
                                                                #1  0x0000
```

问题 1（trace 部分，input 是谁调用的）：backtrace 显示 #0 为 input、#1 为 main，即 input 由 main 调用（main 中 0x401acc 处的 call 0x401977 <input>）。

问题 2（断点位置与前三条指令）：断点停在 input+8 而不是函数首条指令，因为断点设在函数入口，而入口处的前三条指令已经执行完。前三条指令是函数 prologue：push rbp 把调用者（main）的帧指针 rbp 保存到栈上；mov rbp,rsp 把 rsp 赋给 rbp，建立 input 自己的帧指针；sub rsp,0x10 向下移动栈顶 16 字节，为局部变量 expression[16] 分配空间。此刻 rbp 指向的内存中的值 0x721000800ff0 是 main 的 rbp，它来自 main 入口处的 push rbp（0x401ac3）。

问题 3（rbp+8 处的值）：[rbp+8] = 0x401ad1，对照 disas main 可知它是 main 中 call input 的下一条指令（mov $0x0,%eax）的地址，也就是 input 的返回地址。

问题 4（rbp+8 的值从哪来）：在 main 中 call 0x401977 <input>（地址 0x401acc）之前下断点，用 si 逐指令执行。执行到 call 指令时，CPU 硬件自动把下一条指令的地址 0x401ad1 压入栈（rsp 先减 8 再写入），随后把 rip 改为 input 的入口 0x401977。因此写入 rbp+8 处的正是 main 中的这条 call 指令。

问题 5（call、prologue、leave、ret 的作用）：call：把返回地址（下一条指令地址）压栈，然后跳转到被调函数，完成控制流转移并记录返回点；prologue（push rbp; mov rbp,rsp; sub rsp,N）：保存调用者帧指针、建立本函数帧指针、为局部变量分配空间，建立栈帧；leave（mov rsp,rbp; pop rbp）：等价于撤销 prologue——把栈顶收回到帧指针处、恢复调用者的 rbp；ret：从栈顶弹出返回地址赋给 rip，控制流回到调用者。call 与 ret 配对实现函数调用与返回，prologue 与 leave 配对维护栈帧。

二、用不同长度的字符串输入测试 dark-calc

测试 1：输入 20 个字符 'A'（gets 连同结尾 '\0' 共写 21 字节）。expression 只有 16 字节，多出的 5 字节覆盖了 saved rbp 槽的低 5 字节。程序没有崩溃：eval 正常执行 system("echo $((AAAAAAAAAAAAAAAA))")，bash 将未定义变量按 0 计算，输出 0；main 循环再次调用 input，stdin 已到 EOF，gets 返回 NULL，程序打印 answer（42_is_The_nswer）并 exit(0)，退出码为 0。

测试 2：输入 32 个字符（共写 33 字节）。16 字节 expression + 8 字节 saved rbp + 9

字节，返回地址槽的 8 个字节被 0x41（'A'）全部覆盖。input 的 leave 执行后 rbp = 0x4141414141414141，ret 把 0x4141414141414141 弹入 rip，访问非法地址，进程收到 SIGSEGV（Segmentation fault），退出码 139。

在 gdb 中于 input 的 ret（0x4019d6）处断点观察溢出数据的位置：

```
Breakpoint 1, 0x00000000004019d6 in input ()
                                                                                rsp=0x7210
```

可见超量的数据按地址递增方向依次写穿了 expression、saved rbp 槽和返回地址槽——这正是栈上的缓冲区溢出（Stack-based Buffer Overflow）：gets 不检查长度，输入超出缓冲区后继续向高地址写，篡改了紧邻缓冲区的控制数据。

三、任务：利用缓冲区溢出执行 ./malware

攻击流程：

1. 反汇编 input 确定栈布局：expression 位于 rbp-0x10，saved rbp 位于 rbp，返回地址位于 rbp+8，因此从缓冲区开头到返回地址共 24 字节。

2. input 返回前执行 strcpy(command_buf, expression)，把输入原样复制到全局变量 command_buf（0x6030d0）。所以把要执行的命令 "./malware" 放在 payload 开头，它就会成为 system 的参数。

3. 覆盖返回地址为 eval 中 system 调用之前的 0x401a57：

```
401a57: be d0 30 60 00   mov esi,0x6030d0
                                                                                401a5c: bf
```

即跳过 eval 前面拼接 echo $((...)) 字符串的代码，直接执行 system(command_buf)。rdi 由既有代码自行设置，payload 无需注入任何参数。

4. 栈对齐：从 input 的 ret 跳到这里时 rsp 恰好保持 16 字节对齐（与 eval 正常路径的调用点一致），否则会因 glibc 内部的 movaps 对齐检查而崩溃。

5. system("./malware") 启动 malware 脚本；脚本沿父进程链向上查找 dark-calc 进程，确认是 dark-calc 触发的炸弹后打印成功信息。payload 与运行输出：

```
payload = b"./malware\x00" + b"A"*(24-len(cmd)) + struct.pack("<Q", 0x401A57) + b"\n"
```

四、用 GDB 检查 main 函数的机器码

在 main 入口（0x401ac3）断点，用 x/16bx 0x401ac3 检查内存：

```
0x401ac3 <main>:   0x55 0x48 0x89 0xe5 0xb8 0x00 0x00 0x00
                                                                                0x401acb <
```

问题 1（前三行汇编对应的机器码）：push rbp = 55；mov rbp,rsp = 48 89 e5；mov eax,0 = b8 00 00 00 00。

问题 2（内存排列方式）：机器码按指令的先后顺序从低地址到高地址连续存放，一条指令的字节依次紧邻下一条指令；多字节的操作数按小端序（Little Endian）存放，即最低有效字节在最低地址。三条指令分别占据 0x401ac3（1 字节）、0x401ac4 – 0x401ac6（3 字节）、0x401ac7 – 0x401acb（5 字节）。例如 call 0x401977 的机器码 e8 a6 fe ff ff 中，相对偏移为 0xfffffea6 = -0x15a（从 0x401ad1 到 0x401977），小端存放为 a6 fe ff ff：

```
0x401ac3  55          push rbp
                                                                    0x401ac4
```

五、任务：栈空间执行

1. 为什么 dark-calc 的栈地址固定且可执行：该程序用链接器选项 --wrap=main 把 main 重定向到 __wrap_main。程序启动时 init_fixed_stack 调用 mmap(0x721000000000, 0x802000, PROT_READ|PROT_WRITE|PROT_EXEC, MAP_PRIVATE|MAP_ANONYMOUS) 在固定地址建立一块 RWX 内存，并把首尾各一页 mprotect 为 PROT_NONE 作为隔离页；随后 __wrap_main 把 rsp 切换到固定栈顶 0x721000801000 再调用真正的 main。于是 main 及其所有子函数的栈帧都位于这块固定地址的可执行内存上——ASLR 与 NX 在本任务中被绕过。

2. 由此可精确计算：input 的 rbp = 0x721000800fe0，expression = rbp-0x10 = 0x721000800fd0。覆盖返回地址为 0x721000800ff0（缓冲区+32 字节处），并在这里放置 13 字节的 shellcode：

```
50              push rax          ; rsp -= 8，修正 printf 的栈对齐
                                                                    bf 04 00 0
```

3. grade_eval 先把 edi 存入 [rbp-0x4] 再与 4 比较，故 grade==4 成立，执行 printf 输出 "Hope A will be your grade!"。有趣的是 mov edi,4 的立即数字节恰好落在 grade_eval 计算出的 [rbp-0x4] 位置上，值正好是 4。

4. 空间约束：gets 最多能写 48 字节（0x721000801000 起是 PROT_NONE 隔离页），payload 总长必须 ≤47 字节。这里用 push rax（1 字节）代替 sub rsp,8（3 字节）压缩 shellcode，最终 payload 为 24+8+13+1 = 46 字节。

5. shellcode 用 jmp 而非 call，栈上返回地址槽仍保存 shellcode 的地址，grade_eval 的 ret 会跳回 shellcode 循环打印，无害。输出：

```
$ ./dark-calc < payload_dark
                                                                    Enter an e
```

六、任务：ROP

1. 环境：dark-calc-nx 的栈不可执行（GNU_STACK 标志为 RW）且启用 ASLR，但代码段地址固定（非 PIE）。溢出点与偏移与前面相同（24 字节到达返回地址）。

2. 按提示在 0x4019d0 – 0x401ae3（__libc_csu_init 的尾部）寻找 gadget，可用的有：

```
0x401a2b: pop rbp; pop r12; pop r13; pop r14; pop r15; ret
                                                                    0x401a33:
```

区间内没有能直接给 grade_eval 传参的 mov rdi 类 gadget（虽然有 pop rdi; ret，但本攻击选择完全依赖提示范围内的 pop rbp 方案，并利用栈来构造参数，见下）。

3. 攻击思路：不调用 grade_eval 的入口（那里会检查 [rbp-0x4]==4），而是跳入其中段 0x401989（cmp 与条件跳转之后）：

```
401989: b8 45 00 00 00   mov eax,0x45
                                                                    40198e: 2b
```

只要让 [rbp-0x4] == 4，printf 就会打印 'A'。rbp 由 pop rbp 控制，把它指向 .bss 中地址固定的 command_buf+8（0x602098），则 [rbp-0x4] = [0x602094] = command_buf[4..7]。

4. "利用栈来操作"：payload 前 24 字节（padding）的第 4..7 字节放置 04 00 00 00，input 中的 strcpy(command_buf, expression) 会把它复制进全局 command_buf，于是 [0x602094] == 4。参数 4

经由栈写入 .bss，再通过 rbp 间接读出，全程不依赖随机化的栈地址。

5. 栈对齐：跳入 grade_eval 中段跳过了它的 prologue，计算可得 call printf 时 rsp 恰好 16 字节对齐，避免了 glibc 的 movaps 对齐崩溃（实验中如果不对齐，printf 会直接段错误，需要增减 ret 数目修正）。

6. 收尾：grade_eval 的 leave/ret 会把 rsp 转到 rbp（0x602098），pop rbp 与 ret 依次弹出 command_buf[8..15] 与 command_buf[16..23]——这两个位置由 payload 的 padding[8..23] 控制，填入 main 的地址 0x4019b4 即可回到 main，随后 input 读到 EOF 打印 answer 并正常退出。完整 ROP 链：

```
padding = b"AAAA" + b"\x04\x00\x00\x00" + b"B"*8 + p64(0x4019B4)   ; 24 字节
```

链：p64(0x4

七、任务：Canary

1. dark-log 编译时开启了 Stack Protector。反汇编 input 得到栈布局（结构体 log_frame 固定了成员顺序）：

```
rbp-0x120  expression[128]   <- gets 写入
```

rbp-0x0a0

2. gets 恰好写入 260 字节：覆盖 expression、logMsg 和 len（=168），在 canary 之前停住，canary 完好。

3. 关键：input 中的 memcpy(f.logMsg + f.len, f.expression, 128) 的拷贝目标由 len 决定。len 已被篡改为 168，于是拷贝目标 = logMsg+168 = 返回地址槽。128 字节的拷贝把 expression 的内容写到返回地址及更高的位置，全程不触碰 canary——canary 校验通过，但返回地址已被改写。这正是"借助函数自身的代码绕过 Stack Protector"。

4. expression 的前 32 字节组成一条小 ROP 链（dark-log 自身的 gadget），返回后依次执行：

```
0x4018a3  pop rdi; ret    ; rdi = 4
```

0x4018a4 r

grade_eval 打印并 fflush 之后，其自身的 canary 检查因栈上恰好落入链地址而失败，进程被 __stack_chk_fail 终止——但要求的输出已经产生。输出：

```
$ ./dark-log < payload_canary
```

Enter a lo

5. 思考题：如何重新排列这些局部变量使上述攻击失效？——把可溢出的 char 数组（缓冲区）放在最靠近 canary 的位置，把标量（len）放在缓冲区下方（低地址处）。这正是现代编译器默认的重排策略：溢出会先破坏 canary 而不是 len，函数返回前的校验立即调用 __stack_chk_fail 终止程序。更本质的原则：任何越界写都应该先碰到 canary 而非其他控制数据，并且拷贝长度不应取自可能被溢出的字段。

八、测试结果

按课程 CI（lab3-checks.yml）的相同方式在本地复测，四个 payload 全部通过：

```
payload.py               -> attack/dark-calc        You have successfully detonated the bomb!
Congratulations!  PASS
```

payload_da